

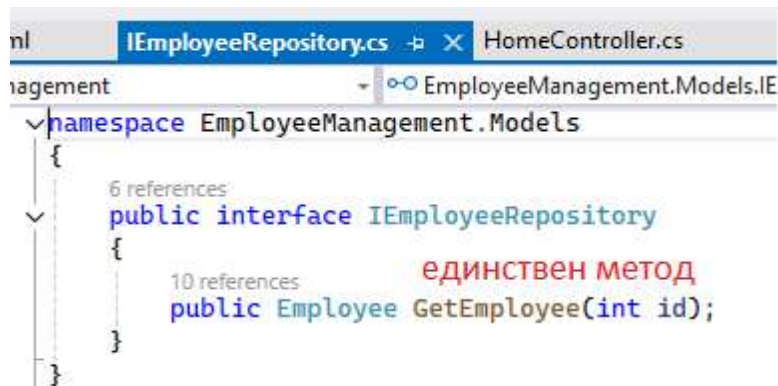
List view

Искаме да изобразим всички служители в една html таблица:

List View in ASP.NET Core MVC

ID	Name	Department
1	Mary	HR
2	John	IT
3	Sam	IT

До момента IEmployeeRepository има само един единствен метод GetEmployee



който връща служител по неговото id.

За да направим метод, който връща всички служители по списък, той трябва да е тип IEnumerable в който вкарваме лист от <Employee> с нова променлива, която наричаме GetAllEmployees:

```

1 using System.Collections.Generic;
2
3 namespace EmployeeManagement.Models
4 {
5     6 references
6     public interface IEmployeeRepository
7     {
8         10 references
9         Employee GetEmployee(int id);
10
11         IEnumerable<Employee> GetAll();
12     }
13 }

```

IEnumerable изисква Generic библиотеката, <Employee> показва вид списък от сложни обекти тип Employee, дори IntelliSense ни подсказва GetAll(); а ние добавяме Employees към името на метода.

```

1 using System.Collections.Generic;
2
3 namespace EmployeeManagement.Models
4 {
5     6 references
6     public interface IEmployeeRepository
7     {
8         10 references
9         Employee GetEmployee(int id);
10
11         0 references
12         IEnumerable<Employee> GetAllEmployees();
13     }
14 }

```

Tooltip for `IEnumerable<Employee>`:
 Exposes the enumerator, which supports a simple iteration over...

Ако дадем Run на приложението, макар да няма никаква грешка в кода, ще хвърли, че новият метод GetAllEmployee не се използва никъде в MockEmployeeRepository:

100 % No issues found

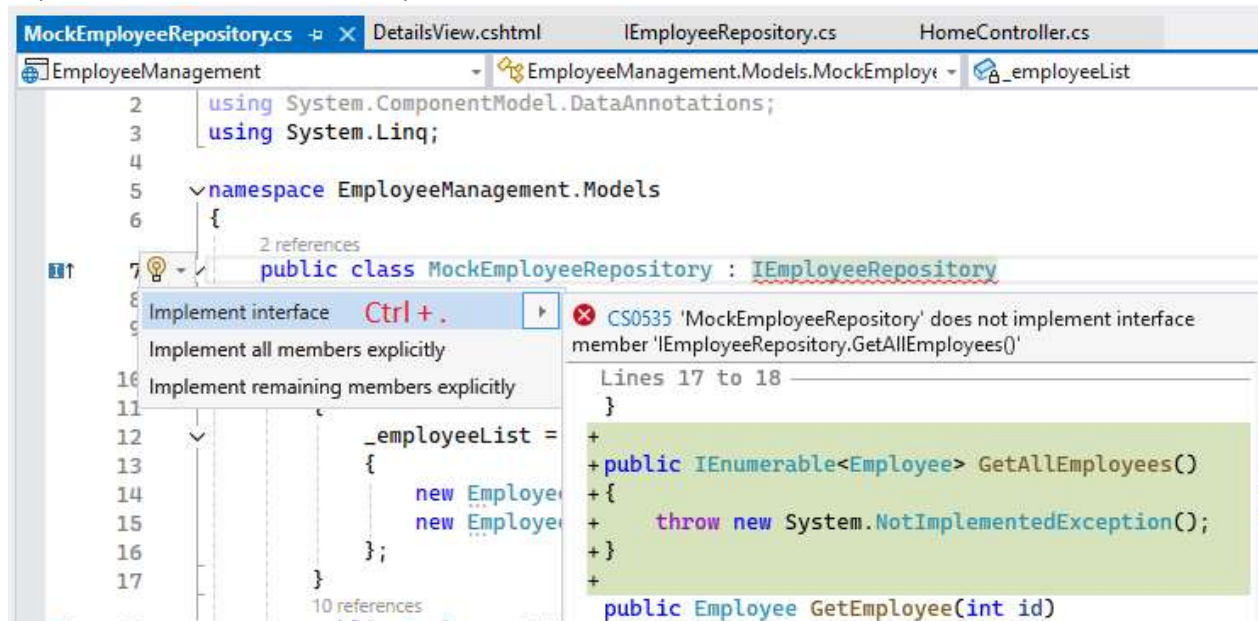
Error List

Entire Solution 1 Error 0 of 6 Warnings 0 of 1 Message

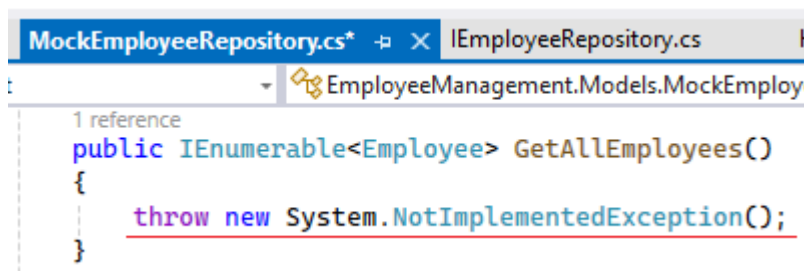
	Code	Description	Project	File
✖	CS0535	'MockEmployeeRepository' does not implement interface member 'IEmployeeRepository.GetAllEmployees()'	EmployeeManag...	MockEmployeeRepository.cs

все още никъде
новият метод не се използва

IEmployeeRepository има само 1 имплементация (изпълнение) и тя е в MockEmployeeRepository, но вече дава грешка в нея с червено подчертаване, защото все още сме й предоставили имплементация само на единия GetEmployee(), а на другия нов метод, който току-що й добавихме GetAllEmployee() липсва. Когато кликнем върху грешката, ни отвежда до кода в MockEmployeeRepository, а когато я коригираме автоматично с Ctrl + . дава най-елементарната корекция с ново изключение, просто за да го използва:



Която се имплементира по-надолу в кода:



Всичко, което искаме този нов метод да направи, е да ни върне списъка от листа на всички служители. Искаме да ни върне променливата с листа, който е hard-code по-нагоре и го предлага автоматично. По-нататък, ако има време, ще направим имплементацията на списъка вместо с hard-code в лист, с извличане на данни от SQL server DB.

```
MockEmployeeRepository.cs* X |EmployeeRepository.cs |HomeController.cs
EmployeeManagement.Models.MockEmployee | GetAllEmployees()

2 references
public class MockEmployeeRepository : IEmployeeRepository
{
    private List<Employee> _employeeList; memory field
    0 references
    public MockEmployeeRepository()
    {
        _employeeList = new List<Employee>()
        {
            new Employee() { Id = 1, Name = "Мария", Family = "Георгиева", Phone = "01
            new Employee() { Id = 25, Name = "Николема", Family = "Панайотова", Phone
        };
    }

    1 reference
    public IEnumerable<Employee> GetAllEmployees()
    {
        return _employeeList; similar implementation
    }

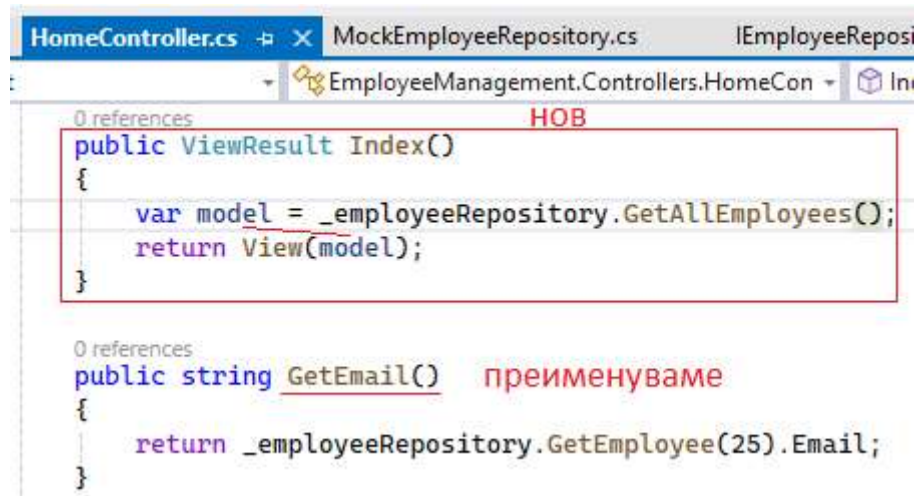
    10 references
    public Employee GetEmployee(int id)
    {
        // return _employeeList[id];
        return _employeeList.FirstOrDefault( e => e.Id == id);
    }
}
```

Т.е. return на метода, за да върне целия списък със служителите.

```
public IEnumerable<Employee> GetAllEmployee()
{
    return _employeeList;
}
```

След като оправихме моделите, остава да ги използваме в контролера – преименуваме стария метод Index -> GetEmail и създаваме нов като страница, която да ни върне всички служители.

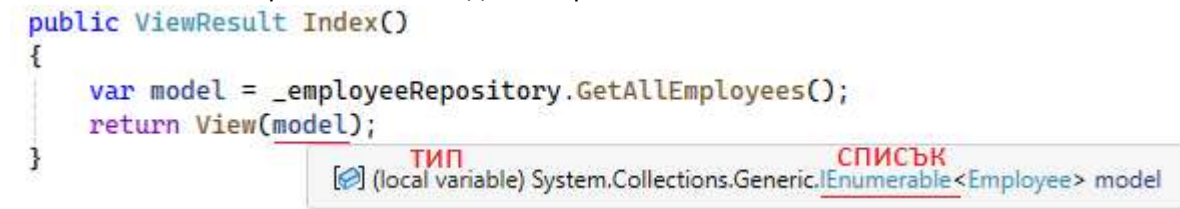
Съхраняваме този лист в нова променлива model.



```
0 references HOB
public IActionResult Index()
{
    var model = _employeeRepository.GetAllEmployees();
    return View(model);
}

0 references
public string GetEmail() преименуваме
{
    return _employeeRepository.GetEmployee(25).Email;
}
```

Искаме вместо 1 потребител с id = 1 да ни върне всички.



```
public IActionResult Index()
{
    var model = _employeeRepository.GetAllEmployees();
    return View(model);
}
```

ТИП (local variable) System.Collections.Generic.IEnumerable<Employee> model СПИСЪК

Когато стартираме приложението, изгледът още не е създаден.

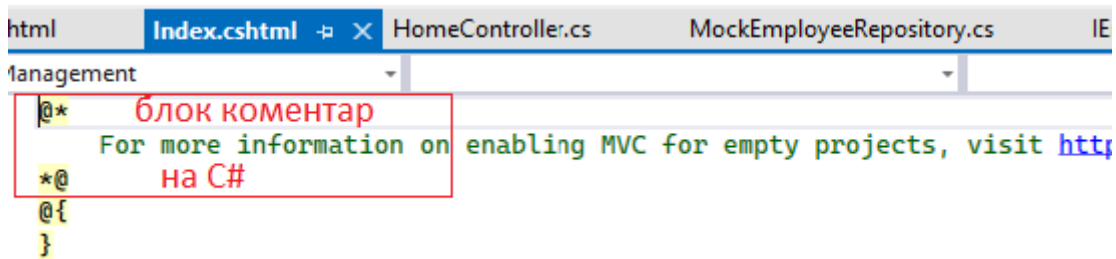
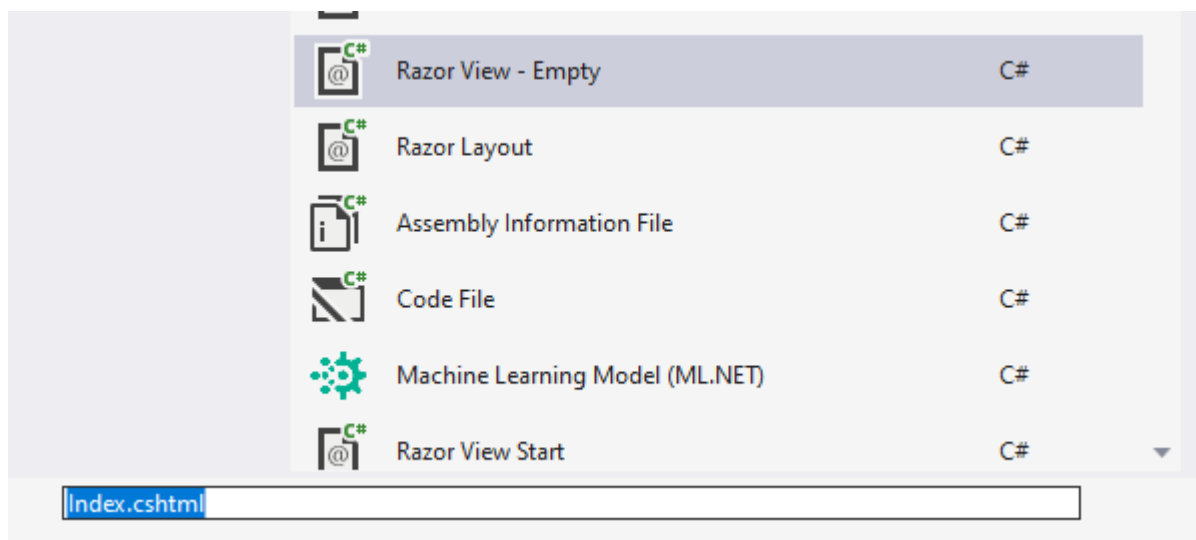
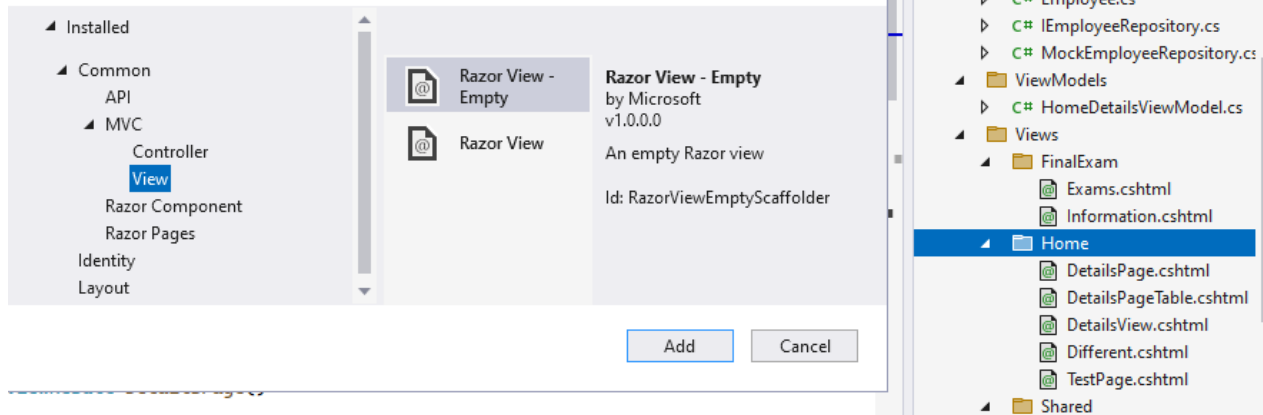


An unhandled exception occurred while processing the request.

InvalidOperationException: The view 'Index' was not found. The following locations were searched:
/Views/Home/Index.cshtml
/Views/Shared/Index.cshtml
/Pages/Shared/Index.cshtml

View изисква IActionResult, а не string, както досега, така че го сменяме. Като string не е имало проблем да бъде изписан в Index() action, но View изисква самостоятелна страница .cshtml.
Home -> Add -> View -> Razor (empty)

Add New Scaffolded Item

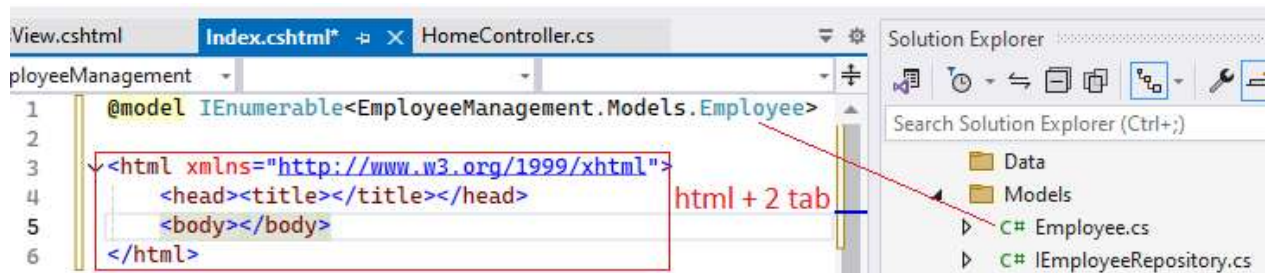


За Intellisense, моделът за нашето View е от IEnumerable.

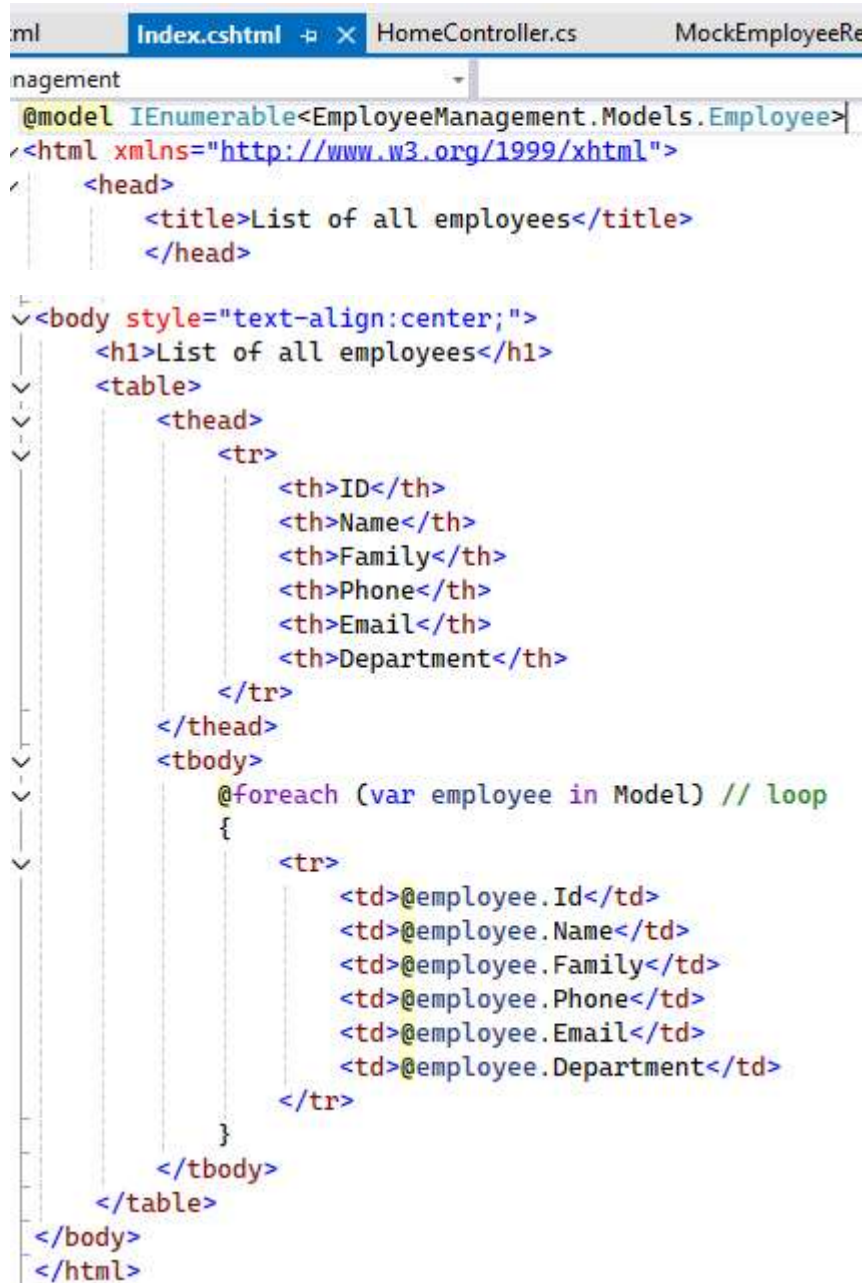
Досега изписвахме `IEnumerable<Employee>`, но понеже се касае за namespace, е необходим пълният адрес:

```
@model IEnumerable<EmployeeManagement.Models.Employee>
```

След което вграждаме базовата html-структура с html + Tab x 2:



Попълваме кода, така че с цикъл да се изведат всички стойности на свойствата за employee:



Вижда се гъвкавостта, която C# предлага – автоматично се генерира нов ред за всеки нов служител:

```
@model IEnumerable<EmployeeManagement.Models.Employee>
<html xmlns="http://www.w3.org/1999/xhtml">
  <head>
    <title>List of all employees</title>
  </head>
  <body style="text-align:center;">
    <h1>List of all employees</h1>
    <table>
      <thead>
        <tr>
          <th>ID</th>
          <th>Name</th>
          <th>Family</th>
          <th>Phone</th>
          <th>Email</th>
          <th>Department</th>
        </tr>
      </thead>
      <tbody>
        @foreach (var employee in Model) // loop
        {
          <tr>
            <td>@employee.Id</td>
            <td>@employee.Name</td>
            <td>@employee.Family</td>
            <td>@employee.Phone</td>
            <td>@employee.Email</td>
            <td>@employee.Department</td>
          </tr>
        }
      </tbody>
    </table>
  </body>
</html>
```

Резултат:



List of all employees

ID	Name	Family	Phone	Email	Department
1	Мария	Георгиева	0123456789	Maria@omg.bg	Biology
25	Николета	Панайотова	9876543210	nikoleta@omg.bg	IT

Интересният момент тук е поредното объркващо повторение на думата на променливата employee, но няма как тя да бъде пренесена от друг документ тук, тя е локална променлива и като такава, можем да я кръстим както си искаме.

Освен това, за да се извика C# код с @Model в съдържанието, очевидно не е задължително те да бъдат слети. Възможно е да има други структури между тях като това условие за цикъл @.....Model

не са слети

```
@foreach (var employee in Model) // loop
```

може друго име

```
<tr>
  <td>@employee.Id</td>
  <td>@employee.Name</td>
  <td>@employee.Family</td>
```

(local variable) EmployeeManagement.Models.Employee employee

Извличайки данните от employee in Model динамично генерираме по 1 нов ред <tr> в цикъла foreach loop, всеки от които с по 6 клетки table data <td>.